DESIGN AND ANALYSIS OF ALGORITHMS

DIVIDE AND CONQUER METHOD

1. Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format:

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format:

First Line Contains Integer – Number of zeroes present in the given array.

ANSWER:

```
#include <stdio.h>
int zeros(int *arr, int m){
   int left = 0, right = m-1, mid;
   while(left <= right){
      mid = (left+right) / 2;
      if(arr[mid] == 1) left = mid+1;
      else right = mid-1;
   }
   return m-left;
}
int main(){
   int n;
   scanf("%d", &n);
   int arr[n];
   for(int i = 0; i < n; i++) scanf("%d", &arr[i]);
```

```
    printf("%d", zeros(arr, n));
    return 0;
}
```

2. Given an array nums of size n, return the majority element.

The majority element is the element that appears more than ⌊n / 2⌋ times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]

Output: 3

Example 2:

Input: nums = [2,2,1,1,1,2,2]

Output: 2

Constraints:

n == nums.length

$1 <= n <= 5 * 10^4$

$-2^{31} <= nums[i] <= 2^{31} - 1$

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

ANSWER:

```
#include <stdio.h>
int majority(int* arr, int n){
    int count = 0, first = arr[0], second;
```

```
    for(int i = 0; i < n; i++){
        if(arr[i] == first) count++;
        else second = arr[i];
    }
    if(count > n/2) return first;
    else return second;
}
int main(){
    int n;
    scanf("%d", &n);
    int arr[n];
    for(int i = 0; i < n; i++) scanf("%d", &arr[i]);
    printf("%d", majority(arr,n));
    return 0;
}
```

3. Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

ANSWER:

```
#include <stdio.h>
int findFloor(int arr[], int low, int high, int x) {
    if (low > high) return -1;
```

```
    if (arr[low] > x) return -1;
    if (arr[high] <= x) return arr[high];
    int mid = low + (high - low) / 2;
    if (arr[mid] == x) return arr[mid];
    if (arr[mid] < x) {
        if (mid == high || arr[mid + 1] > x) return arr[mid];   // catch it here
        return findFloor(arr, mid + 1, high, x);
    } else {
        if (mid > low && arr[mid - 1] <= x) return arr[mid - 1];
        return findFloor(arr, low, mid - 1, x);
    }
}
int main(){
int n;
scanf("%d",&n);
int arr[n];
for(int i=0;i<n;i++)scanf("%d",&arr[i]);
int x;
scanf("%d",&x);
int result=findFloor(arr,0,n-1,x);
printf("%d\n",result);
return 0;
}
```

4. Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as “No”.

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value “x”)

ANSWER:

```
#include <stdio.h>
int found = 0;
void findPair(int arr[], int low, int high, int x) {
   if (low >= high) {
      return;
   }
   int sum = arr[low] + arr[high];
   if (sum == x) {
      printf("%d\n", arr[low]);
      printf("%d\n", arr[high]);
      found = 1;
      return;
   } else if (sum < x) {
      findPair(arr, low + 1, high, x);
   } else {
      findPair(arr, low, high - 1, x);
   }
}
```

```
int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    int x;
    scanf("%d", &x);
    findPair(arr, 0, n - 1, x);
    if (!found) {
        printf("No\n");
    }
    return 0;
}
```

5. Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

ANSWER:

```
#include <stdio.h>
#include <stdlib.h>
int compare(const void *a, const void *b)
{
   return *(int *)a - *(int *)b;
}
int main(){
   int n;
   scanf("%d", &n);
   int arr[n];
   for(int i = 0; i < n; i++) scanf("%d", &arr[i]);
   qsort(arr, n, sizeof(int), compare);
   for(int i = 0; i < n; i++) printf("%d ", arr[i]);
   return 0;
}
```